MRV Assistant — Product Requirements Document

The Source of Truth | Version 4.0 | April 2026

PART 1 — CONTEXT & PEOPLE

Who This Is For

Adetutu Owoeye (Adetutu) is a remote Clinical Records Specialist subcontracted by Love Olaleye, who holds the primary contract with Olive Health & Wellness Clinic in Greenville, TX. The clinic is contracted with VES (Veterans Evaluation Services — a Maximus company) to support the U.S. Department of Veterans Affairs in processing Compensation & Pension (C&P) disability claims for veterans.

The team shares a single login. Love, Adetutu, and other subcontractors all log into VES using Love's credentials. This is the current working arrangement.

Adetutu's role in plain language: A veteran files a disability claim with the VA. The VA sends the veteran's entire medical history — sometimes 15,000 pages — to VES. A VES doctor must review these records and fill out Disability Benefits Questionnaires (DBQs) about specific conditions (e.g. "Does this veteran have a service-connected knee condition?"). Adetutu's job is to prepare those records for the doctor by finding the relevant pages before the doctor ever opens the file. She is the person who does the pre-work so the doctor can do their job efficiently.

The tool we are building is FOR ADETUTU. The beneficiary is the VES doctor.

Her Exact Daily Workflow (Current — Without Extension)

1. Log into VES Provider Portal using shared credentials
2. Click the purple C-file icon to open Maximus MRV in a new tab
3. A patient file loads — anywhere from 500 to 15,000+ pages
4. Note what DBQ type is being evaluated (e.g. "Hip and Thigh", "Spine")
5. Open the Search panel in MRV left sidebar
6. Type medical keywords one at a time into Text Search (e.g. "MRI", "surgery", "spine", "x-

ray") and press Enter for each one

7. The system highlights matching words across all pages in cyan
8. She uses the ◄ ► navigation arrows to jump between highlighted pages
9. On each page, she reads context around the highlight to judge relevance
10. If relevant, she manually creates a sticky note on that page
11. She types a brief note describing the finding
12. Repeat for every relevant page across potentially hundreds of hits
13. At the end, the doctor uses the sticky note panel to jump to flagged pages
14. Repeat this entire process for multiple patients per day

The bottleneck: Steps 8-12. This is pure repetitive scrolling and clicking. On a 3,291-page file with 286 search hits, this could take 1-3 hours manually.

What "Next Week Is Our Busiest Week" Means

The team message said: "Next week is going to be our busiest day ever this month. Plan accordingly." This is real — Adetutu is under real workload pressure. The extension must not create ANY new work. Not one extra click. Not one thing to clean up. If it fails, it must fail silently and gracefully, not leave a mess.

PART 2 — THE TOOL WE ARE BUILDING

What It Is

A Chrome Extension called **MRV Assistant** that:

1. Reads which keywords are already searched in Maximus
2. Uses the MRV search API to get the complete list of ALL pages with hits
3. Scrolls to each relevant page automatically
4. Places a colour-coded sticky note on that page
5. Shows Adetutu a summary when done

Result: Adetutu opens the Sticky Notes panel in Maximus and jumps directly to every flagged page. What took hours takes minutes.

What It Is NOT

- Not a general AI browser agent
- Not something that reads or extracts medical text
- Not something that requires Adetutu to write code, JSON, or scripts
- Not something that modifies the MRV portal UI
- Not something that stores patient data
- Not something that requires internet beyond what MRV already uses

The One-Sentence Promise

"Search your keywords in Maximus, click one button, come back to flagged pages."

PART 3 — CONFIRMED TECHNICAL FACTS

Everything in this section was confirmed via live DOM inspection sessions. Nothing here is a guess.

The MRV Viewer Architecture

- URL: `https://viewer.ves-prod.maxfedsolutions.com/viewer.html`
- Rendering: Pure SVG. No iframe, no canvas, no PDF.js.
- The entire document is one giant `<svg>` inside `#viewpane`
- All pages are laid out vertically at absolute Y coordinates
- Only ~11 pages are loaded in the DOM at any time (lazy loading)
- Scrolling is achieved by changing the `transform` attribute of `g.container`

Confirmed DOM Selectors

Element	Selector	Notes
Document SVG root	`#viewpane > svg`	Single SVG, whole document
Page groups	`g[id^="P_"]`	Format: `P_{DCN}_{pageNum}`

Scroll container	`g.container`	Has `transform="translate(0,-Y)"`
Search hit groups	`g[id^="sr_"]`	ALL hits — DBQ scope + text search
Note create button	`g[id^="createNoteButton"]`	Dynamic timestamp in ID
Color swatches	`rect.noteColorSwitchPatch`	Inside createNoteButton
Notes container	`g.stickynotes`	Parent of all placed notes
Existing notes	`g.stickynote`	Individual placed notes
Search chip list	`ul.search_chips`	In left sidebar
Active search terms	`ul.search_chips .search_term`	Text of each chip
Left sidebar	`#filterPanel`	358px wide when open

Confirmed Swatch Colors (exact hex)

Color	Hex	Our Usage
Yellow	`#fdfdb8`	HIGH PRIORITY — strong match to DBQ
Orange	`#ff9000`	REVIEW NEEDED — weaker match
Green	`#75f367`	Not used in v1
Cyan	`#55f5ff`	Not used in v1
Pink	`#ffc7ff`	Not used in v1
Hot pink	`#ff2789`	Not used in v1

Confirmed Scroll Mechanism

Page positions confirmed from live data:

- Page 636: `translate(0, 370249.58)` — Y = 370,249
- Page 637: `translate(0, 371576.64)` — Y = 371,576

- Page height: ~1,327 SVG units
- Container scroll: `translate(0, -379354.53)` — negative Y = scroll position

To scroll to a page:

```
targetY = pageGroup.getAttribute('transform') → parse Y value
newContainerY = -(targetY) + (svgHeight / 2)
g.container.setAttribute('transform', `translate(0, ${newContainerY})`)
```

After writing transform, poll every 100ms (max 20 attempts) to confirm the target page `g#P_{DCN}_{pageNum}` has rendered into viewport before placing note. If page never appears: skip, log to failedPages, continue.

Why direct transform write, not WheelEvent: WheelEvent fires the handler but may be ignored by Maximus's custom scroll logic. Direct attribute write on `g.container` is guaranteed to work because Maximus reads this attribute to determine what to render.

Confirmed Search API

```
POST https://ves-viewer-2-prod-endpoint.ves-prod.maxfedsolutions.com/gateway/prox

Headers:
  Authorization: Bearer {JWT}  ← captured by intercepting fetch()
  Content-Type: application/json

Body:
  {
    "query_stmt": "*spine*",
    "query_type": "terms",
    "queryOp": "OR",
    "slop": 0,
    "fuzzy": 0,
    "allow_stemming": false
  }
```

The term is wrapped in wildcards: `spine` → `*spine*`. The API returns all matching pages across the FULL document (not just loaded pages). Response format: unknown — we never saw the response body. **This is why we need a fallback:** if API response parsing fails, fall back to DOM-only extraction using currently loaded `g[id^="sr_"]` elements.

Confirmed Note Placement Mechanism

1. Scroll container to target page using transform write

2. Wait for page to be in viewport (300–500ms depending on speed setting)
3. Find `g[id^="createNoteButton"]`
4. Find `rect.noteColorSwitchPatch` with matching fill colour
5. Dispatch `MouseEvent('click', {bubbles:true, cancelable:true, view:window})` — MouseEvents work regardless of `isTrusted` value (confirmed by research) — Only KeyboardEvents are typically gated by isTrusted
6. Poll every 100ms for new `g.stickynote` to appear (max 50 attempts = 5s)
7. When note appears, find `textarea` inside it
8. Set content using React nativeInputValueSetter trick:

```
const setter = Object.getOwnPropertyDescriptor(
  window.HTMLTextAreaElement.prototype, 'value'
).set;
setter.call(textarea, content);
textarea.dispatchEvent(new Event('input', {bubbles: true}));
textarea.dispatchEvent(new Event('change', {bubbles: true}));
```

9. Wait 100ms, then dispatch blur event to trigger save
10. Wait between notes (speed-dependent: 500ms normal, 1000ms slow, 300ms fast)

Confirmed User Guide Facts (from official MRVUserGuide.pdf)

- Gold star on pages = DBQ Scope match (auto-filtered by condition type)
- Purple bookmark tags = pages relevant to selected DBQs
- Blue eye = previously viewed page
- `g[id^="sr_"]` captures BOTH DBQ scope hits AND text search hits
- Chips require real human keystrokes (isTrusted=true) — extension cannot add chips
- User must add keywords in Maximus manually BEFORE running extension
- Session has a time limit — user can relaunch MRV to continue
- Sticky notes persist server-side (survive page reload)
- Keywords DO NOT persist after page reload (must be re-entered)
- Multi-page side-by-side view: 2-20 pages visible simultaneously

PART 4 — WHAT THE EXTENSION MUST DO

Core Flow (Happy Path)

```
User opens Maximus → adds keywords → highlights appear →
opens extension popup → clicks Run →
extension places notes → user reviews summary →
opens Sticky Notes panel → jumps to flagged pages → done
```

Stage 1 — Token Capture (runs silently on page load)

The content script wraps `window.fetch` immediately on injection to intercept the Bearer JWT token from the first search API call Maximus makes naturally. This token is stored in memory for use in our own API calls.

```
const origFetch = window.fetch;
window.fetch = async (...args) => {
  const url = typeof args[0] === 'string' ? args[0] : args[0]?.url;
  const res = await origFetch.apply(this, args);
  if (url?.includes('esearch') && args[1]?.headers?.Authorization) {
    window._mrvToken = args[1].headers.Authorization;
  }
  return res;
};
```

Stage 2 — Document Scan (on Run click)

1. Find all `g[id^="P_"]` elements → parse pageNum and translateY per page
2. Find all `g[id^="sr_"]` elements → map to pages using translateY comparison
3. Group hits by page → build list of {pageNum, translateY, hitCount, keywords}
4. Sort by pageNum ascending
5. Filter by user threshold (min hits per page — default 1)

Stage 3 — Full Hit List via API (if token available)

If `window._mrvToken` exists:

- For each active search term from `ul.search_chips .search_term`

- POST to esearch API with that term
- Parse response → extract all page numbers
- Merge with DOM-found pages
- Deduplicate by page number
- If API fails: log warning, continue with DOM-only results, show warning to user

Stage 4 — Note Placement

For each target page (sorted ascending):

1. Scroll to page using transform write
2. Wait for DOM settle
3. Determine colour: yellow if high relevance, orange if standard
4. Click swatch → wait for note → set content → blur → wait between notes
5. Report progress to background

Stage 5 — Summary

Show in popup:

- Notes placed successfully
- Notes failed (page list)
- Pages skipped (already had notes)
- Total time elapsed
- Instruction: “Open Sticky Notes panel in Maximus to jump to flagged pages”

PART 5 — NOTE CONTENT FORMAT

Maximus sticky note textarea is small. Content must be brief.

Yellow Note (High Priority)

```
⭐ HI-PRI
KW: MRI | Surgery
Pg {pageNum}
```

```
{MM/DD/YYYY}
```

Orange Note (Review)

```
⚠ REVIEW
KW: Spine | X-ray
Pg {pageNum}
{MM/DD/YYYY}
```

Rules

- Maximum 4 lines
- Keywords truncated to 2 max with "..." if more
- Date is the day the extraction ran
- No patient name stored — HIPAA compliance

Relevance Logic (High vs Review)

High Priority (yellow) if page category matches DBQ type:

DBQ Type	High Priority Categories
Hip and Thigh	orthopedic, hip, thigh, surgery, radiology, mri, xray, ct, operative, medrep
Spine / Back	spine, lumbar, thoracic, cervical, disc, neuro, mri, xray, ct, radiology
Shoulder / Arm	shoulder, rotator, bicep, surgery, radiology, mri, xray, orthopedic
Knee / Leg	knee, leg, meniscus, acl, pcl, surgery, radiology, orthopedic
Mental Health	psychology, psychiatry, ptsd, behavioral, counseling, mental health
Hypertension	cardiology, cardiac, heart, bp, blood pressure, primary care
Diabetes	endocrine, diabetes, glucose, hba1c, primary care, lab
Hearing Loss	audiology, ent, hearing, audiogram
Respiratory	pulmonary, lung, copd, asthma, radiology, ct, xray
General	ALL pages get yellow (no filter — doctor decides)

Page category comes from the page header text: `DCN {n} Pg {n} {Category}`. Parsed from the `text` element inside each `g[id^="P_"]` group.

PART 6 — THE POPUP UI

Screen States (only one visible at a time)

State 1: READY — document detected, keywords found, ready to run **State 2: NO DOCUMENT** — not on MRV or no file open **State 3: NO KEYWORDS** — MRV open but no search chips active **State 4: CONFIRMING** — >20 notes, asking user to confirm **State 5: RUNNING** — progress bar active, stop button visible **State 6: DONE** — results summary shown **State 7: RESUMING** — previous run interrupted, offer to resume

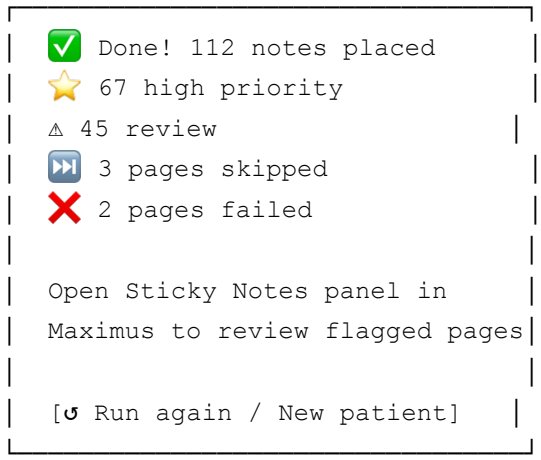
Popup Layout

```
┌──────────────────────────────────┐
│  🏥 MRV Assistant                │
│  PACE, G · 3,291 pages           │
│  286 hits found · SPINE active   │
├──────────────────────────────────┤
│  DBQ Type: [Hip and Thigh  ▼]    │
│                                  │
│  Min hits per page: [1 ▼]        │
│                                  │
│  Speed: [Normal ▼]               │
│                                  │
│  ☑ Skip pages with notes         │
├──────────────────────────────────┤
│  [⚡ Run Highlight Extraction]   │
└──────────────────────────────────┘
```

Running state:

```
┌──────────────────────────────────┐
│  Placing note 47 of 112...       │
│  ████████████░░░░  42%           │
│  Est. 8 min remaining            │
│  [🔲 Stop]                       │
└──────────────────────────────────┘
```

Done state:



Settings (persist in chrome.storage.local)

- DBQ Type dropdown
- Min hits threshold (1–5)
- Speed setting (Slow / Normal / Fast)
- Skip existing notes checkbox (default: ON)

PART 7 — ARCHITECTURE

Files

```
mrv-assistant/
├── manifest.json          ← MV3 extension manifest
├── background.js          ← Service worker: state, badge, keepalive
├── content.js             ← Injected into MRV: token capture, extraction, note p
├── popup.html             ← Extension popup UI
├── popup.js               ← Popup logic: reads state, sends commands
└── icon.png               ← Extension icon (48×48)
```

Message Flow

```
content.js → chrome.runtime.sendMessage → background.js
                                                ↓ stores in chrome.storage.local
popup.js ← chrome.storage.local.get ← (polls on open + storage.onChanged)
popup.js → chrome.tabs.sendMessage → content.js (for RUN / STOP commands)
```

Why this architecture:

- Popup closes when Adetutu clicks elsewhere — port connections die
- `chrome.storage.local` persists state regardless of popup lifecycle
- Background service worker stays alive with keepAlive ping during active runs
- Content script runs independently of popup — extraction continues if popup closes

State Object (in chrome.storage.local)

```
{
  "mrvState": "idle | running | done | error",
  "mrvProgress": { "current": 47, "total": 112, "pct": 42 },
  "mrvResults": { "placed": 112, "highPri": 67, "lowPri": 45, "skipped": 3, "fail
  "mrvPatient": "PACE, G",
  "mrvStartTime": 1714176000000,
  "mrvKeywords": ["spine", "mri", "surgery"],
  "mrvDbq": "hip",
  "mrvSpeed": "normal",
  "mrvShouldStop": false,
  "mrvFailedPages": [312, 847]
}
```

PART 8 — TIMING STRATEGY

Network speed and computer performance vary across the team's laptops and Wi-Fi connections. All timing is multiplied by a speed factor.

Action	Fast (0.6×)	Normal (1×)	Slow (2×)
After scroll to page	180ms	300ms	600ms
Poll for note to appear	60ms interval	100ms interval	200ms interval
Max wait for note	3s	5s	10s
After textarea set	60ms	100ms	200ms
After blur (save)	180ms	300ms	600ms
Between notes	300ms	500ms	1000ms

Keepalive ping	every 20s	every 20s	every 20s

Default: Normal. If notes land on wrong pages → switch to Slow. If extraction feels too fast and notes are not appearing → switch to Slow.

PART 9 — EDGE CASES & ERROR HANDLING

Pre-run Checks (before anything happens)

Check	Fail Condition	Message Shown
On correct URL	Not on maxfedsolutions.com	"Open a patient C-file in Maximus first"
Document loaded	No `g[id^="P_"]` in DOM	"No document detected. Open a C-file"
Keywords active	`ul.search_chips` empty	"No keywords found. Search your terms in Maximus first"
Not already running	`mrvState === 'running'`	Show progress of current run

During Extraction

Situation	Behaviour
Page not found in DOM when scrolling	Skip page, log to failedPages, continue
Swatch not found	Try yellow fallback, if still not found skip page
Note never appears after 5s	Skip page, add to failedPages, continue
Textarea not found	Skip page, add to failedPages, continue
API call fails (401, 500, network)	Fall back to DOM-only, show warning banner
API response format unrecognised	Fall back to DOM-only, show warning banner

User clicks Stop	Finish current note, then stop cleanly
MRV tab reloads mid-run	Content script stops, state saved, resume shown on reopen
Session timeout in MRV	Notes stop appearing — user must relaunch MRV, then resume
Mixed veteran file (multiple DCNs)	Warn user: "Multiple DCNs detected in this file. Proceeding with all pages."
Duplicate note on same page	Detected by counting `g.stickynote` before and after — if count unchanged, treat as failure
>20 notes	Show confirmation: "About to place X notes on Y pages. Continue?"
0 notes needed	Show: "No pages meet your criteria. Try lowering the minimum hits or checking your keywords."

Post-Run

Situation	Behaviour
Some notes failed	Show failed page list: "2 pages could not be noted: 312, 847"
All notes failed	Show: "No notes were placed. Check that Maximus is open and visible, then try Slow speed"
Partial run stopped by user	Show: "Stopped after X notes. Resume? [Yes] [Start fresh]"

PART 10 — WHAT THE EXTENSION CANNOT DO

These are hard limits. Setting correct expectations prevents frustration.

1. **Cannot read the text content of highlighted pages.** The document is rendered as a pixel image inside SVG. There is no selectable text. Notes say WHICH KEYWORD matched on WHICH PAGE, not what the surrounding sentence says. Adetutu still reads each flagged page herself.
2. **Cannot add search keywords automatically.** Maximus requires real human keystrokes to create a search chip. Adetutu must type and press Enter herself before running the extension.

3. **Cannot run if keywords are not already searched.** The extension reads chips that already exist — it does not create them.
4. **Cannot guarantee pixel-perfect page targeting in side-by-side view.** When Maximus shows 4–6 pages simultaneously, the note lands on the page at the top of the visible viewport. This is accurate to ±1 page, which is still vastly better than manual scrolling.
5. **Cannot recover a MRV session that has timed out.** If the session expires, Adetutu relaunches MRV and clicks Resume in the extension popup.
6. **Cannot store or transmit patient data.** All state is local to Chrome on the current machine and is cleared on Chrome close.

PART 11 — WHAT PREVIOUS VERSIONS GOT WRONG (DO NOT REPEAT)

v1 — Built without PRD

- No PRD at all — jumped straight to code
- Used wrong selector (`g.citation-tokens` which was always empty)
- No scroll mechanism — notes all landed on the current page
- No timing strategy — ran too fast on slow networks

v2 — PRD existed but had critical bugs

- Popup messaging broken (MV3 popup closes on focus loss)
- WheelEvent scroll unreliable (replaced with direct transform write in v4)
- ensureKeywordsSearched() used wrong selector and was never called
- parseTranslateY regex failed on negative values and spaces
- No guard against running twice

v3.1 — Most bugs fixed but selector still wrong

- Switched to `g[id^="sr_"]` (correct) but `g.citation-tokens` still in fallback
- Speed setting added but timing still too aggressive on Normal
- API fallback not implemented — API failures caused complete failure

- Confirmation gate existed but threshold was hardcoded

v4 (THIS VERSION) — Fixes all known issues

- Correct selector: `g[id^="sr_"]` only, no fallback to wrong selector
- Correct scroll: direct `g.container` transform write
- Correct messaging: chrome.storage.local as source of truth, popup polls
- Correct timing: speed multiplier applied to all waits
- Correct API usage: try/catch with DOM fallback
- Correct regex: handles negatives and spaces
- Confirmation gate user-controlled (threshold 1-5)
- Resume after interruption
- Failed pages shown in results

PART 12 — DEFINITION OF DONE

The extension is complete and ready to hand to Adetutu when:

□ It installs from a zip file without errors
□ It shows the correct patient name and page count when Maximus is open
□ It shows “Search your terms first” when no chips are active
□ It places notes on the correct pages (verified on 3 pages minimum)
□ Notes contain the correct content format
□ Progress updates correctly while popup is open
□ Progress survives popup being closed and reopened
□ Stop button works cleanly
□ Results summary shows accurate counts
□ It does NOT crash or leave garbage if something fails
□ A non-technical person can use it with zero instructions beyond “click Run”

PART 13 — INSTALL INSTRUCTIONS (for Adetutu)

1. Download the `mrv-assistant.zip` file
2. Unzip it — you get a folder called `mrv-assistant`

3. Open Chrome → go to `chrome://extensions`
4. Turn on **Developer mode** (toggle, top right)
5. Click **Load unpacked** → select the `mrv-assistant` folder
6. The 🏥 icon appears in the Chrome toolbar

To use:

1. Open Maximus, open a patient C-file
2. Search your keywords in Maximus as normal (so highlights appear)
3. Click the 🏥 icon in Chrome toolbar
4. Select the DBQ type
5. Click **Run Highlight Extraction**
6. Wait for it to finish (watch the progress bar)
7. Open **Sticky Notes** panel in Maximus to jump to flagged pages

If notes land on wrong pages: Set Speed to **Slow** and run again on a new patient.

End of PRD v4.0 This document is the single source of truth. Do not build anything not described here. Do not add features not described here. Do not modify selectors without re-confirming via DOM probe.